Physical-Device Evaluation of a Potential TOCTOU in the iOS MTE Tagging Path on iPhone 17e / iOS 26.5.1

Kaya Ercihan

16.06.2026

Abstract

Target: **iPhone 17e / iOS 26.5.1**, physical device. The candidate helper is the routine reverse-mapped in the supplied iOS 26.5 graph as `irg_mte_function`/ `vm_memtag_generate_and_store_tag()` at `0xFFFFFE000A8578A4` [6]. The application displays this mapping, but the run does not directly trace helper execution. The static candidate is an ordering question around `IRG` followed by `STG`/`ST2G`/`STGM`. Prior path-sensitive analysis did not identify a complete barrier-, lock-, or acquire/release-based publication protocol on several zone paths [5, 6]. Same-cache Skywalk allocation and free are excluded from final TOCTOU pressure because their MTE critical sections are serialized by `skm_sl_lock`.

The stock-userland harness therefore uses zone-only parallel pressure and retains Skywalk as a sequential reachability and noise-control path. The run produced `12443103` overlap-counter increments and no externally observed MTE tag-fault signature in the captured Console evidence. Each increment means that a userland worker phase sampled the opposite phase as active; it does not establish helper execution, distinct physical-core placement, same-element reuse, or `STG`-to-`vm_memtag_load_tag()` overlap. The main limitation is control granularity: the candidate relation is instruction-level, while the harness reaches the allocator only through Mach/BSD syscalls and intervening IPC, allocator, policy, and lock paths. Result: the mapped syscall surface and concurrent workload are demonstrated; no tag fault, same-object stale-tag observation, or exploitability was demonstrated.

1 Target and hypothesis

Table 1: Concrete target

Field	Value
Physical target	iPhone 17e
iOS target	iOS 26.5.1
Kernel helper	`irg_mte_function`, reverse-mapped in the supplied iOS 26.5 graph as `vm_memtag_generate_and_store_tag()`
Helper address	`0xFFFFFE000A8578A4` in the supplied iOS 26.5 graph; displayed by `MTETrigger.app`, not directly observed through runtime tracing
Userland harness	`MTETrigger.app`, C backend exposed to SwiftUI through `MTETrigger-Bridging-Header.h`
Relevant surfaces	Mach IPC / zone allocator (parallel TOCTOU pressure); BSD socket / Skywalk slab allocator (sequential reachability/noise control only; same-cache allocation/free serialized by `skm_sl_lock`)
Fault criteria	External kernel/process evidence of `TAG_FAULT`, a tag-check fault, `DFSC 0x11`, MTE-derived `EXC_BAD_ACCESS`, or an MTE-derived panic; guidance text emitted by `MTETrigger.app` is not itself fault evidence

Hypothesis: a tag store in one execution context can be followed by pointer publication, tag reload, or tagged access in another execution context before a sufficient happens-before relationship between the tag store and that consumer is established [5, 6]. This is a CWE-362 style shared-resource ordering question, not a standalone claim that `IRG` or `STG` is incorrect [3, 8]. A positive result requires an external MTE fault, an allocator-consistency symptom, or a same-object kernel trace linking retagging to later use; userland phase overlap alone is pressure evidence.

Evidence boundary. This chapter dynamically measures syscall execution and overlap between userland allocation and teardown phases. It does not observe kernel element addresses, allocation tags, helper entry or exit,

or the CPU ID executing the helper. Claims about the helper and caller chain are derived from the preceding static correlation [5, 6]; claims about the physical run are limited to application counters, screenshots, Console captures, and externally visible fault behavior.

2 Static path basis

The helper performs logical tag generation and allocation-tag stores. The reduced instruction sequence below is the relevant primitive: no explicit architectural barrier is present inside the helper body, so any required ordering must come from callers, locks, or an acquire/release protocol [5, 6].

```
FFFFFE000A8578A4: BTI     c
FFFFFE000A8578A8: IRG     X0, X0, X2
FFFFFE000A8578AC: AND     X9, X0, #0xFFFFFFFFFFFFFFF0
FFFFFE000A8578B0: ADD     X8, X1, X0
FFFFFE000A8578B4: ADD     X10, X8, #0xF
FFFFFE000A8578B8: AND     X8, X10, #0xFFFFFFFFFFFFFFF0
FFFFFE000A8578BC: SUB     X11, X10, X9
FFFFFE000A8578C0: CMP     X11, #0x3F
...
FFFFFE000A8578D4: STG     X9, [X9],#0x10
FFFFFE000A8578FC: STGM    X11, [X9]
FFFFFE000A857934: ST2G    X9, [X9],#0x20
FFFFFE000A857978: ST2G    X9, [X9],#0x20
FFFFFE000A857988: ST2G    X9, [X9],#0x20
FFFFFE000A857974: RET
```

Listing 1: MTE helper core at `0xFFFFFE000A8578A4`: tag generation and tag stores

Table 2: Statically mapped trigger routes and runtime roles

Route	Userland trigger	Mapped runtime role
Zone init/refill	`mach_port_allocate()`, optional `mach_msg()` pressure	Dynamically exercised by the physical-device harness; static correlation maps the syscall path to zone expansion/refill and `zcram_memtag_init()` when the selected zone is tag-enabled. Used for allocation-side pressure.
Zone free/retag	`mach_port_deallocate()` plus `mach_port_mod_refs(..., RECEIVE, -1)`	Dynamically exercised by the physical-device harness; static correlation maps the teardown path to `zone_tag_free_element()` and free-side retagging when the selected zone is tag-enabled. Selected as the primary dynamic candidate.
Skywalk alloc/free	loopback `socket()`, `sendmsg()`, `recv()`, `close()`	Dynamically exercised by the physical-device harness as a sequential reachability path; static correlation maps the socket path to Skywalk slab MTE code when the relevant tagging gate is enabled. Same-cache allocation/free MTE critical sections are serialized by `skm_sl_lock`; retained for reachability and noise comparison.

3 Allocator-specific assessment

Zone allocator. The interesting pattern is free-side retagging and allocation-side tag load separated by zone-lock boundaries. The source-level model used by the harness is shown in Listing 2. The pressure target is same-class object lifetime churn, not direct tag selection.

```
// Free side: retag before the later zone-lock transition.
static inline caddr_t zone_tag_free_element(zone_t zone,
    caddr_t addr, vm_size_t elem_size)
{
    if (__improbable((uintptr_t)addr > 0xFF00000000000000ULL))
        return addr;
    addr = vm_memtag_generate_and_store_tag(addr, elem_size);
    zone_tag_element(zone, addr, elem_size);
    return addr;
}

// Allocation return side: tag is loaded after zone_unlock().
zone_unlock(zone);
return zalloc_return(zone, addr, flags, esize); // vm_memtag_load_tag()
```

Listing 2: Reduced zone model used for the race schedule

Skywalk serialization in XNU. The slab path is useful for reachability/noise checks but not for final TOCTOU pressure. XNU wraps both the public alloc/free entry points and the locked bodies with the per-cache mutex `skm_sl_lock`; the relevant MTE helper call therefore executes while the lock is held on both sides [1, 4, 7].

```
/* bsd/skywalk/mem/skmem_cache_var.h */
#define SKM_SLAB_LOCK(_skm)     lck_mtx_lock(&(_skm)->skm_sl_lock)
#define SKM_SLAB_UNLOCK(_skm)   lck_mtx_unlock(&(_skm)->skm_sl_lock)

```

```
/* bsd/skywalk/mem/skmem_slab.c */
int skmem_slab_alloc(struct skmem_cache *skm, ...)
{
    SKM_SLAB_LOCK(skm);
    err = skm->skm_slab_alloc(skm, oi, oim, skmflag);   // alloc_locked
    SKM_SLAB_UNLOCK(skm);
}

SKM_SLAB_LOCK_ASSERT_HELD(skm);
tagged_address = vm_memtag_generate_and_store_tag(
    bc->bc_addr, skm->skm_objsize);                     // alloc side

void skmem_slab_free(struct skmem_cache *skm, void *buf)
{
    SKM_SLAB_LOCK(skm);
    skm->skm_slab_free(skm, buf);                       // free_locked
    SKM_SLAB_UNLOCK(skm);
}

SKM_SLAB_LOCK_ASSERT_HELD(skm);
tagged_addr = vm_memtag_generate_and_store_tag(
    buf, skm->skm_objsize);                             // free side
```

Listing 3: Condensed Skywalk locking and MTE call sites

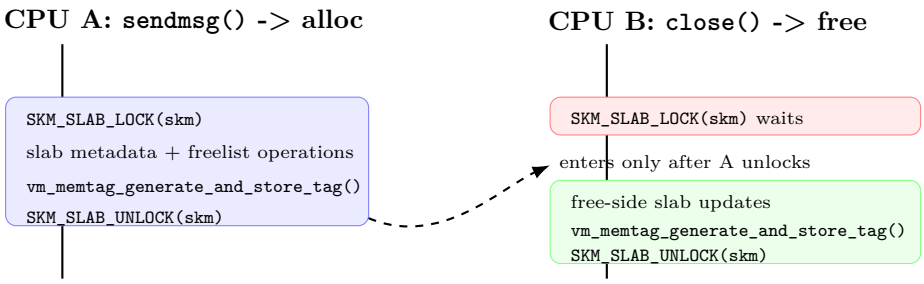


Figure 1: Same-cache Skywalk alloc/free MTE critical sections cannot overlap. The mutex orders slab-state stores and tag stores before the next entrant runs; this is critical-section serialization, not a literal global cache flush.

This is why the application keeps Skywalk out of the final parallel trigger:

```
void mte_trigger_parallel(uint32_t num_workers, uint32_t duration_ms,
    uint32_t num_ports, uint32_t num_sockets)
{
    (void)num_sockets; /* Skywalk removed: slab lock serializes MTE ops */
    ...
    mte_log("[Parallel] Skywalk excluded (slab lock serializes MTE ops)");
}
```

Listing 4: Skywalk exclusion in the parallel trigger

4 Trigger implementation

The C API exposes sequential zone/Skywalk paths, the zone-only parallel trigger, a stop function, fault-filter output, and precondition checks. The parallel trigger uses separate allocation and teardown workers, alternating Darwin affinity tags, a mutex-guarded port ring, and atomic userland phase-state counters [7]. The counters characterize the behavior of the harness; they do not observe kernel helper execution, element identity, physical-core placement, or tag-store/load timing.

```
typedef void (*mte_log_callback_t)(const char *message);
void mte_set_log_callback(mte_log_callback_t callback);
void mte_trigger_zone_path(uint32_t num_ports);
void mte_trigger_skywalk_path(uint32_t num_sockets, uint32_t send_size);
void mte_trigger_parallel(uint32_t num_workers, uint32_t duration_ms,
    uint32_t num_ports, uint32_t num_sockets);
void mte_trigger_parallel_stop(void);
void mte_print_fault_filter_guide(void);
void mte_check_preconditions(void);

static _Atomic int32_t  g_active_allocs = 0;
static _Atomic int32_t  g_active_frees = 0;
static _Atomic uint64_t g_overlap_count = 0;
```

Listing 5: Public trigger API and overlap counters

```
// Alloc worker
int32_t concurrent_frees = atomic_load(&g_active_frees);
atomic_fetch_add(&g_active_allocs, 1);
if (concurrent_frees > 0) atomic_fetch_add(&g_overlap_count, 1);
zone_phase_alloc_ports(task, ports, np);
atomic_fetch_sub(&g_active_allocs, 1);

// Free worker
int32_t concurrent_allocs = atomic_load(&g_active_allocs);
atomic_fetch_add(&g_active_frees, 1);
if (concurrent_allocs > 0) atomic_fetch_add(&g_overlap_count, 1);
zone_phase_destroy_ports(task, batch.ports, batch.count);
atomic_fetch_sub(&g_active_frees, 1);
```

Listing 6: Userland zone phase-entry overlap counter

Counter semantics. `g_overlap_count` increments at most once per worker phase entry when the worker's pre-entry sample observes the opposite phase as active. It does not measure overlap duration or unique overlap intervals. Because each worker samples the opposite phase before marking its own phase active, nearly simultaneous starts can go uncounted if both workers observe the opposite counter as zero before either active counter is incremented. Conversely, the sampled opposite phase can end before the current worker marks itself active, so a near-boundary handoff can still be counted. The counter is therefore an approximate userland workload-concurrency indicator, not an exact overlap count or a count of kernel races.

5 Userland timing limitation

The trigger cannot call `irg_mte_function` directly. From stock userland the request first enters the kernel through Mach or BSD syscalls and then traverses IPC/socket handling, object lookup, reference-count updates, allocator selection, zone/slab policy checks, and finally the MTE-enabled allocation or free path. The tested app therefore controls only allocator pressure and object lifetime churn; it does not control the exact instruction boundary at which `IRG` and `STG`/`ST2G`/`STGM` execute.

The relevant race window is much narrower than the syscall path that reaches it. The interesting interval is the helper-local publication interval after the allocation-tag store is issued and before the same object, or tag-dependent state derived from it, is consumed by another execution context through the allocation return path, especially around `vm_memtag_load_tag()`. That interval is an instruction-level/inter-core visibility window, not a userland scheduling window. On a physical device it is expected to be a very short microarchitectural ordering interval, potentially on the order of core/coherency latencies, while the userland trigger can influence it only indirectly after crossing a Mach/BSD syscall, IPC or socket processing, allocator logic, tagging gates, and relevant lock paths.

This provides a plausible explanation for the negative dynamic result. The reported `12443103` counter increments show that allocation and teardown phase entries frequently observed the opposite userland phase as active on separate worker threads, but they do not prove same-element reuse at the exact `STG`-to-`vm_memtag_load_tag()` boundary. The app can increase transition rate and bias execution toward concurrent, potentially cross-core activity; it cannot deterministically align a free-side retag of a specific zone element with a concurrent allocation-side tag load of that same element inside the required instruction-level interval. Conclusive validation therefore requires kernel-side instrumentation or a controlled kernel/debug environment that records the element address, CPU ID, helper entry/exit, free-side retag, allocation-side tag load, and subsequent use or fault outcome for the same object.

Arm defines `IRG` as the instruction that generates and inserts a logical address tag, while `STG`, `ST2G`, and `STGM` perform allocation-tag stores. The instruction-set reference specifies the architectural behavior of these instructions but does not define their implementation latency on the A19 CPU. Accordingly, the cycle range below is treated as a working timing estimate rather than an architectural guarantee or a measurement from the tested device [2, 3].

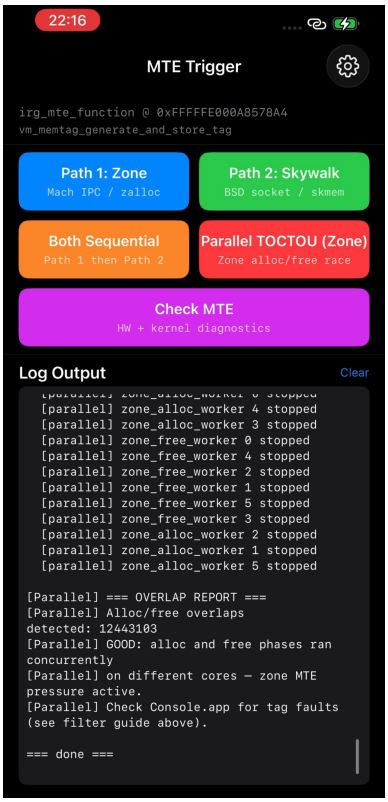
Table 3: Why userland phase overlap is not same-object kernel evidence

Layer	Limitation on a physical stock device
Target window	Working estimate: the same-object visibility interval after `STG`/`ST2G`/`STGM` and before the corresponding `vm_memtag_load_tag()` or tagged access is hypothesized to be on the order of single-digit to tens of cycles, i.e. nanosecond-class at GHz clock rates. This interval was not measured on the A19 implementation and is not an architectural latency guarantee.
Caller path	`mach_port_allocate()` and teardown must traverse a trap, IPC code, `zalloc`/`zfree`, allocator policy, and return; the mapped helper edge is reached only deep inside this path.
Scheduling	Affinity tags only bias placement. They neither prove execution on two distinct physical cores nor schedule workers at a chosen instruction boundary around `irg_mte_function`.
Object control	Userland can increase same-zone churn but cannot force the exact element released by one worker to be the immediate allocation returned to another worker.
Observed counter	`g_overlap_count` records phase-entry observations in which a worker sampled the opposite phase as active. It supports the conclusion that the harness generated concurrent userland pressure, but it does not measure overlap duration, unique overlap intervals, same-element reuse, or `STG -> vm_memtag_load_tag()` overlap.

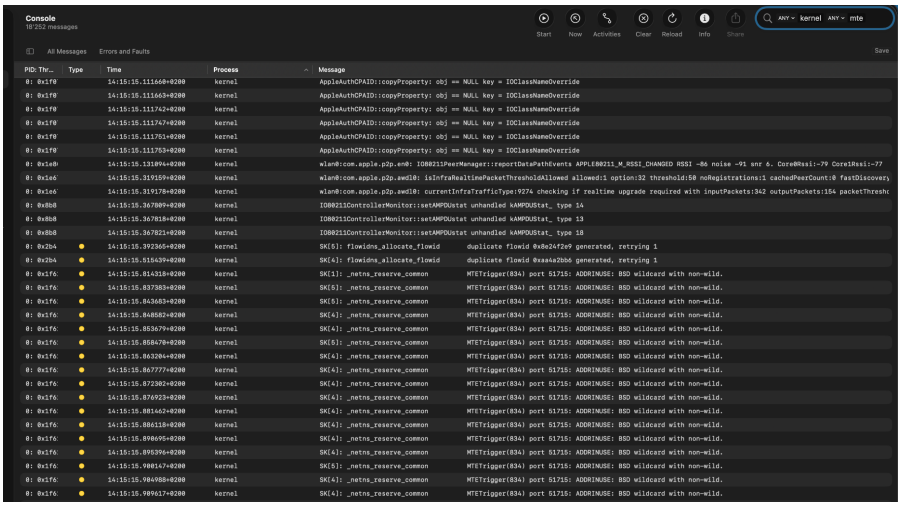
```
Direct helper call:  no; only syscall-mediated reachability
Mapped chain:        user -> Mach/BSD syscall -> IPC/socket -> allocator -> MTE helper
Race target:         STG visibility vs. later vm_memtag_load_tag / tagged access
Window estimate:     instruction-scale, plausibly nanosecond-class; not measured on A19
Measured overlap:    phase overlap only, not same-element instruction overlap
Practical meaning:   same-object helper overlap remains unresolved without kernel instrumentation
```

Listing 7: Timing limitation for the stock-userland trigger

6 Run evidence



App-side overlap report.



Console filter around kernel / MTETrigger rows.

Figure 2: Runtime artifacts from the same run. Left: the physical-device screenshot shows the zone-only parallel mode, the statically mapped helper label/address displayed by the application, and an overlap-counter value of `12443103`. The application's "different cores" line reflects the intended multi-worker/affinity configuration and is not based on captured kernel CPU IDs. Right: Console rows show kernel/network namespace noise, including repeated `ADDRINUSE` lines, rather than an externally generated MTE-fault signature.

Interpretation of the application log. The quoted "on different cores" message is application-generated. The run recorded separate worker threads and alternating affinity tags, but did not record the CPU ID executing `irg_mte_function`. Likewise, the displayed helper address is application metadata derived from the static mapping, not a runtime trace.

```
[Parallel] === OVERLAP REPORT ===
[Parallel] Alloc/free overlaps detected: 12443103
[Parallel] GOOD: alloc and free phases ran concurrently
[Parallel] on different cores - zone MTE pressure active.

_netns_reserve_common MTETrigger(...) port 51715: ADDRINUSE:
    BSD wildcard with non-wild.
bind: Can't assign requested address
```

Listing 8: Overlap result and representative non-MTE noise

7 Result matrix

Table 4: Dynamic result on **iPhone 17e / iOS 26.5.1**

Check	Result
Target mapping	`irg_mte_function`/ `vm_memtag_generate_and_store_tag()` at `0xFFFFFE000A8578A4` from the supplied iOS 26.5 reverse-mapping. The application displays this mapping; no runtime helper-entry trace was captured.
Dynamically exercised routes	Mach IPC / zone allocation and teardown, together with the sequential Skywalk socket path, were executed on the physical device. Their attribution to the corresponding kernel MTE paths is based on the static caller-path correlation.
Parallel zone workload	Achieved. The application recorded `12443103` `g_overlap_count` increments during the parallel zone run.
Counter interpretation	Each increment records a worker phase entry whose pre-entry sample observed the opposite phase as active. It is an approximate userland concurrency metric, not a measurement of overlap duration, a unique race interval, physical CPU placement, or same-object reuse.
Timing limitation	Stock userland cannot call `irg_mte_function` directly; each attempt traverses a Mach trap, IPC handling, allocator selection, zone policy, and the allocation or teardown path. The target same-object tag-store/load relation is treated as instruction-scale and plausibly nanosecond-class, but its absolute duration was not measured on the A19 implementation.
Final candidate surface	Zone allocator only. Same-cache Skywalk allocation/free MTE critical sections are serialized by `skm_sl_lock`; Skywalk is retained for sequential reachability and noise comparison.
Fault-observation calibration	No known-positive MTE fault was generated during this campaign. The negative fault classification is therefore limited to the captured console, crash, and panic-report channels.
Mach-port guard noise	Earlier `EXC_GUARD` class issues are not MTE faults; the corrected teardown path uses `mach_port_deallocate()` and `mach_port_mod_refs(..., RECEIVE, -1)`.
Socket/network noise	Visible `ADDRINUSE` and `bind` errors are socket-path noise, not MTE evidence.
Positive MTE fault	Not observed in the captured run evidence: no external `TAG_FAULT`, tag-check fault, `DFSC 0x11`, MTE-derived `EXC_BAD_ACCESS`, or panic was identified.
Security interpretation	The physical-device syscall workloads and parallel zone pressure are demonstrated. Runtime execution of `irg_mte_function` remains inferred from the static mapping and the required tagging preconditions; same-object tag-store/load overlap, a stale-tag race, and exploitability were not demonstrated by this run.

8 Next validation stages

The immediate follow-up broadens stock-device testing with additional zone-pressure modes and syscall-entry timing perturbation. These experiments extend black-box coverage but cannot prove same-object helper overlap. Conclusive validation still requires either a real external MTE fault or kernel-side tracing of one element through retagging, allocation return, and later `vm_memtag_load_tag()` or tagged access.

References

[1] Apple Inc. XNU allocator and Skywalk source excerpts. Apple Open Source, 2026. Source-level reference points used for reverse mapping of zone tagging and Skywalk slab locking. Files include osfmk/vm/vm_memtag.h, osfmk/kern/zalloc.c, bsd/skywalk/mem/skmem_slab.c, and bsd/skywalk/mem/skmem_cache_var.h.

[2] Arm Limited. IRG: Insert random tag. Arm Instruction Set Reference Guide, Document 100076, Issue 0100. Accessed 2 September 2026 during manuscript revision. URL: `https://support.arm.com/documentation/100076/0100/A64-Instruction-Set-Reference/A64-General-Instructions/IRG`.

[3] Arm Limited. *Arm Architecture Reference Manual for A-profile Architecture*, 2026. Document DDI 0487, Issue M.a; architectural reference for MTE instructions IRG, GMI, STG, ST2G, and STGM, and barrier instructions DMB, DSB, and ISB. URL: `https://developer.arm.com/documentation/ddi0487/maa`.

[4] Kaya Ercihan. Dynamic MTE trigger run log: Physical iPhone 17e / iOS 26.5.1. Research artifact for Report 02, June 2026. Run date: 16 June 2026; includes application counters, Console triage, timing notes, and implementation fixes.

[5] Kaya Ercihan. Investigating a potential TOCTOU in the iOS 26.0.1 MTE tagging path. Technical Report Report 00, Independent Security Research, March 2026. 1 March 2026.

[6] Kaya Ercihan. iOS 26.5 MTE tagging paths: Caller–trigger correlation and TOCTOU preconditions. Technical Report Report 01, Independent Security Research, June 2026. 11 June 2026.

[7] Kaya Ercihan. MTETrigger iOS application source revision used for report 02. Independent research artifact, June 2026. Files include mte_trigger.c, mte_trigger.h, and MTETrigger-Bridging-Header.h; tested 16 June 2026.

[8] MITRE. CWE-362: Concurrent execution using shared resource with improper synchronization. CWE Weakness Catalog. Race-condition taxonomy. URL: `https://cwe.mitre.org/data/definitions/362.html`.